What's Next

- Boolean and Comparison Instructions
- Conditional Jumps
- Conditional Loop Instructions
- Conditional Structures
- **Application: Finite-State Machines**
- Conditional Control Flow Directives

Application: Finite-State Machines

- A finite-state machine (FSM) is a graph structure that changes state based on some input. Also called a state-transition diagram.
- We use a graph to represent an FSM, with squares or circles called nodes, and lines with arrows between the circles called edges.

Application: Finite-State Machines

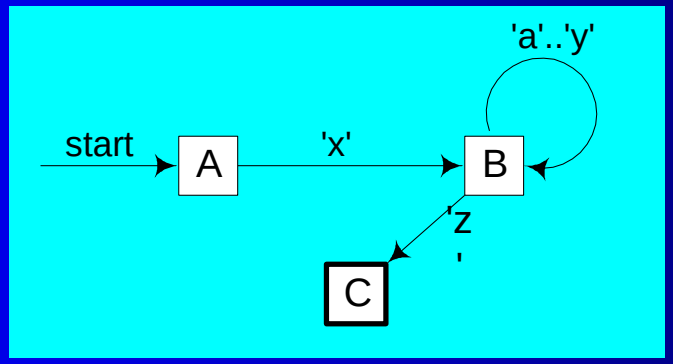
- A FSM is a specific instance of a more general structure called a directed graph.
- Three basic states, represented by nodes:
  - Start state
  - Terminal state(s)
  - Nonterminal state(s)

Finite-State Machine

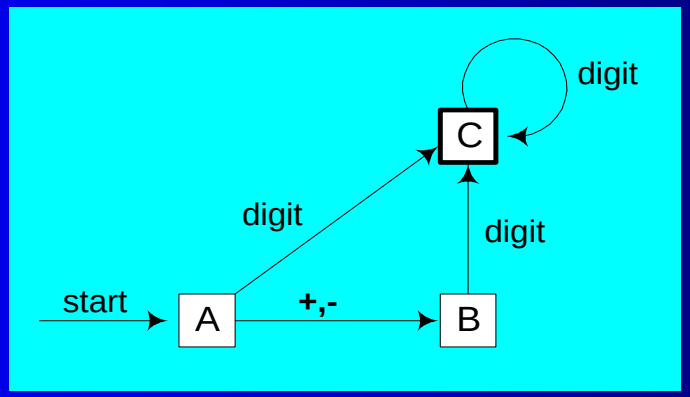
- Accepts any sequence of symbols that puts it into an accepting (final) state
- Can be used to recognize, or validate a sequence of characters that is governed by language rules (called a regular expression)
- Advantages:
  - Provides visual tracking of program's flow of control
  - Easy to modify
  - Easily implemented in assembly language

Finite-State Machine Examples

- FSM that recognizes strings beginning with 'x', followed by letters 'a'..'y', ending with 'z':

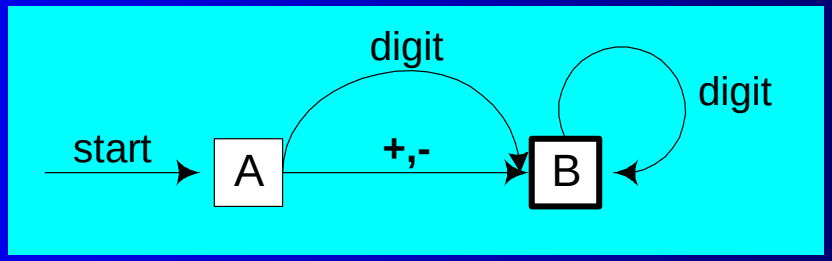


- FSM that recognizes signed integers:



Your Turn . . .

- Explain why the following FSM does not work as well for signed integers as the one shown on the previous slide:



Implementing an FSM

The following is code from State A in the Integer FSM:

```
StateA:
    call Getnext              ; read next char into AL
    cmp al,'+'                ; leading + sign?
    je StateB                 ; go to State B
    cmp al,'-'                ; leading - sign?
    je StateB                 ; go to State B
    call IsDigit              ; ZF = 1 if AL = digit
    jz StateC                 ; go to State C
    call DisplayErrorMsg      ; invalid input found
    jmp Quit
```

View the Finite.asm source code.

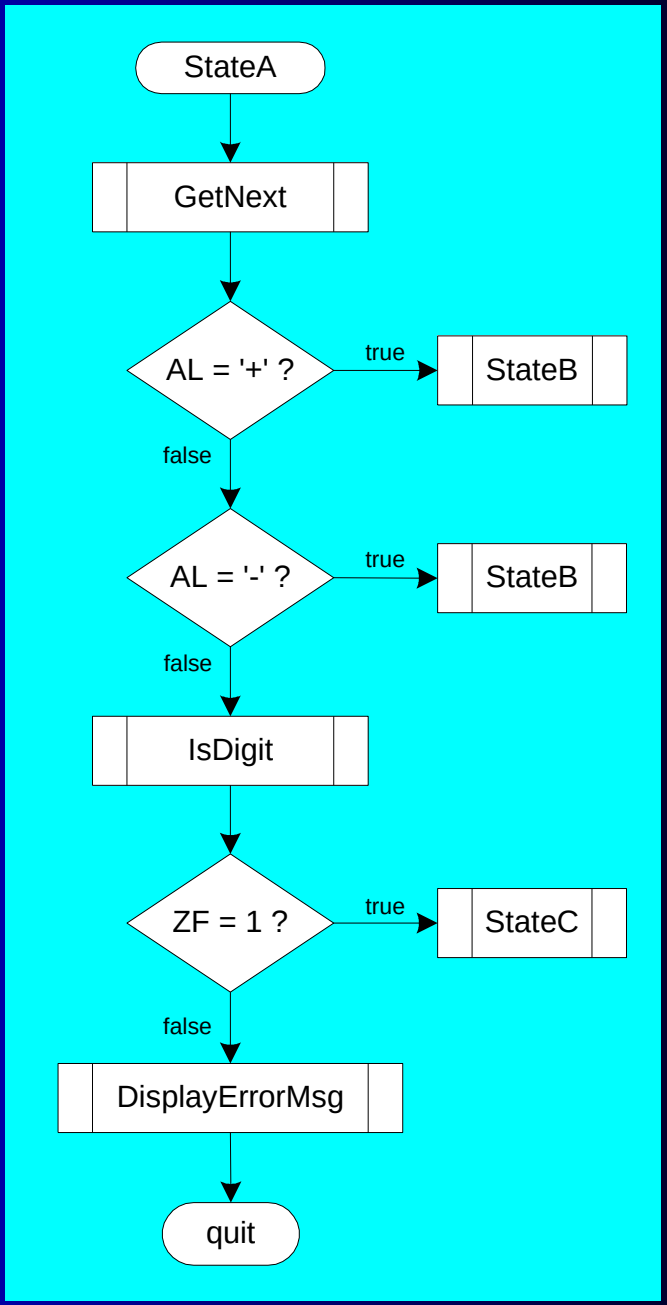
IsDigit Procedure

Receives a character in AL. Sets the Zero flag if the character is a decimal digit.

```
IsDigit PROC
    cmp   al,'0'          ; ZF = 0
    jb    ID1
    cmp   al,'9'          ; ZF = 0
    ja    ID1
    test  ax,0            ; ZF = 1
ID1: ret
IsDigit ENDP
```

Flowchart of State A

State A accepts a plus or minus sign, or a decimal digit.



Full Implementation…

```
TITLE Finite State Machine                    (Finite.asm)
INCLUDE Irvine32.inc
.data
ENTER_KEY = 13
InvalidInputMsg BYTE "Invalid input",0dh,0ah,0

.code
main PROC
    call Clrscr

StateA:
    call  Getnext                ; read next char into AL
    cmp   al,'+'                 ; leading + sign?
    je    StateB                 ; go to State B
    cmp   al,'-'                 ; leading - sign?
    je    StateB                 ; go to State B
    call  Isdigit                ; ZF = 1 if AL contains a digit
    jz    StateC                 ; go to State C
    call  DisplayErrorMsg        ; invalid input found
    jmp   Quit
```

Full Implementation… (cont.)

```
StateB:
    call  Getnext                ; read next char into AL
    call  Isdigit                ; ZF = 1 if AL contains a digit
    jz    StateC
    call  DisplayErrorMsg        ; invalid input found
    jmp   Quit

StateC:
    call  Getnext                ; read next char into AL
    jz    Quit                   ; quit if Enter pressed
    call  Isdigit                ; ZF = 1 if AL contains a digit
    jz    StateC
    cmp   AL,ENTER_KEY           ; Enter key pressed?
    je    Quit                   ; yes: quit
    call  DisplayErrorMsg        ; no: invalid input found
    jmp   Quit

Quit:
    call Crlf
    exit
main ENDP
```

Full Implementation… (cont.)

```
Getnext PROC
     call  ReadChar              ; input from keyboard
     call  WriteChar             ; echo on screen
     ret
Getnext ENDP

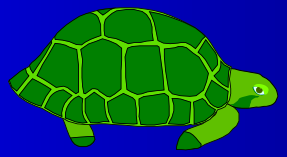
DisplayErrorMsg PROC
     push  edx
     mov   edx,OFFSET InvalidInputMsg
     call  WriteString
     pop   edx
     ret
DisplayErrorMsg ENDP
END main
```

Your Turn . . .

- Draw a FSM diagram for hexadecimal integer constant that conforms to MASM syntax.
- Draw a flowchart for one of the states in your FSM.
- Implement your FSM in assembly language. Let the user input a hexadecimal constant from the keyboard.

Summary

- Bitwise instructions (AND, OR, XOR, NOT, TEST)
  - manipulate individual bits in operands
- CMP – compares operands using implied subtraction
  - sets condition flags
- Conditional Jumps & Loops
  - equality: JE, JNE
  - flag values: JC, JZ, JNC, JP, ...
  - signed: JG, JL, JNG, ...
  - unsigned: JA, JB, JNA, ...
  - LOOPZ, LOOPNZ, LOOPE, LOOPNE
- Flowcharts – logic diagramming tool
- Finite-state machine – tracks state changes at runtime



4C 6F 70 70 75 75 6E